SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4303

GRADED MARK:92/100

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. **(L2)**

Assessment Tool: Data Interpretation (Medium)

Weightage: 20%

QUESTIONS: **Total Marks: 20**

Code of Conduct:

I SOFIA RV - 192511055(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: SOFIA RV

1: HTTP Response Analysis

A web server returns the following HTTP response header:

HTTP/1.1 200 OK

Date: Mon, 01 Apr 2026 10:00:00 GMT

Content-Type: text/html Content-Length: -150 Connection: keep-alive

Questions:

1. **Identify any incorrect or invalid fields in the header.**
2. **Analyze the issue with Content-Lengthand explain its impact.**
3. **Determine whether the status code matches the response correctness.**
4. **Suggest corrections for the identified errors.**

1.1 Identify any incorrect or invalid fields in the header.

The response contains one clearly invalid field and one field with a date inconsistency.

- Content-Length: -150: Invalid because Content-Length must be a non-negative decimal integer representing the message body length in bytes.
- Date: The weekday is incorrect. April 1, 2026, falls on a Wednesday, not a Monday. The correct value is Wed, 01 Apr 2026 10:00:00 GMT.
- HTTP/1.1 200 OK: A valid status line indicating that the request was successfully handled.
- Content-Type: text/html: Valid; it identifies the response body as HTML.
- Connection: keep-alive: Valid in HTTP/1.1, although persistent connections are already the default behavior unless otherwise specified.

1.2 Analyze the issue with Content-Length and explain its impact.

The field Content-Length: -150 is invalid because the value cannot be negative.

The Content-Length header specifies the number of bytes in the HTTP response body. For example, if the HTML response body contains 150 bytes, the correct field is:

Content-Length: 150

Impact of the error:

1. The response violates HTTP message-framing requirements.
2. A strict HTTP implementation may reject the response or treat it as malformed.
3. The browser or intermediary may fail to process the response correctly.
4. Incorrect message framing can cause communication errors between the server and client.

The length must match the actual number of bytes in the transmitted body, not the number of characters in the source code.

1.3 Determine whether the status code matches the response correctness.

The status code 200 OK indicates that the server successfully handled the request. However, it does not guarantee that every response header is valid.

In this example, the status code is not necessarily incorrect merely because another header is malformed. A server could send a 200 OK status line with an invalid Content-Length field.

Therefore, the response has a valid success status code but an invalid header field. The complete response cannot be considered correctly formatted until the header error is fixed.

1.4 Suggest corrections for the identified errors.

Corrected response header:

HTTP/1.1 200 OK

Date: Wed, 01 Apr 2026 10:00:00 GMT

Content-Type: text/html

Content-Length: 150

Connection: keep-alive

Here, 150 is an illustrative value; the actual Content-Length must equal the number of bytes in the response body.

Conclusion: HTTP response headers provide essential information about the content and transmission of a response. Correct header values ensure reliable communication, proper message framing, and compatibility between clients and servers.

2: DOM Structure Analysis

Consider the following DOM tree representation:

<html>

<head>

<title>TestPage</title>

</head>

<body>

** **

** HelloWorld**

** **

</body>

</html>

Questions:

1. **Identify missing or improperly closed elements.**
2. **Analyze how the browser will interpret this structure.**
3. **Determine the impact on rendering and layout.**
4. **Suggest corrections to make the DOM valid.**

2.1 Identify missing or improperly closed elements.

The following element is missing its explicit closing tag:

- — the paragraph element is not explicitly closed with .

The , <body>, <head>, and <html> elements have closing tags. The <title> element is also properly closed.

The corrected paragraph structure should be:

```
<div>
   <p>HelloWorld</p>
</div>
```

2.2 Analyze how the browser will interpret this structure.

HTML browsers use parsing rules to construct a Document Object Model (DOM) from the HTML source. They attempt to recover from common markup errors rather than immediately stopping.

In this example, the HTML parser will generally infer the end of the paragraph before closing the . As a result, the resulting DOM will normally contain a paragraph inside the , even though the source omits .

The browser may therefore display the content without reporting a visible error. However, relying on automatic error recovery is not recommended because malformed markup can make the source harder to maintain and debug.

2.3 Determine the impact on rendering and layout.

- Text rendering: HelloWorld will generally appear as paragraph text.
- Spacing: The paragraph usually has default margins that affect the space around the text.
- DOM structure: Browser error recovery generally creates the expected paragraph-within-div structure in this example.
- Maintainability: Missing closing tags make the source harder to read and can cause unexpected results when additional elements or CSS rules are introduced.
- Validation: The markup may fail HTML validation because the paragraph is not explicitly closed.

Thus, the browser may render the page correctly, but valid and well-structured HTML remains important for predictable behavior.

2.4 Suggest corrections to make the DOM valid.

Corrected HTML:

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <title>TestPage</title>
</head>
<body>
   <div>
      <p>HelloWorld</p>
   </div>
</body>
</html>
```

Conclusion: The DOM represents a webpage as a hierarchical tree of elements. Properly closing elements improves document validity, readability, accessibility, and predictable rendering across browsers.

3: GET vs POST in Login System

A login system uses two different methods:

Method	URL Example	Data Visibility	Security Level
GET	/login?user=admin&pass=1234	Visible in URL	Low
POST	/login	Hidden in body	Moderate

Questions:

1. **Interpret the differences in data transmission between GET and POST.**
2. **Analyze which method is more suitable for login and why.**
3. **Identify security risks in using GET for authentication.**
4. **Suggest best practices for secure login implementation.**

Given methods:

Feature	GET	POST
URL example	/login?user=admin&pass=1234	/login
Data transmission	Query parameters in URL	Usually in the request body
Data visibility	May appear in history, logs, and URLs	Not normally displayed in the URL
Typical purpose	Retrieve resources	Submit data for processing
Suitability for login	Not recommended for credentials	Preferred for submitting credentials

3.1 Interpret the differences in data transmission between GET and POST.

GET method: The client sends data as query parameters appended to the URL. For example:

GET /login?user=admin&pass=1234 HTTP/1.1

Host: example.com

The credentials appear in the URL, where they may be stored in browser history, server logs, and other systems that record request URLs. GET is primarily intended for retrieving resources.

POST method: The client typically sends form data in the request body rather than in the URL.

POST /login HTTP/1.1

Host: example.com

Content-Type: application/x-www-form-urlencoded

user=admin&pass=1234

The credentials are not normally displayed in the address bar. POST is commonly used to submit data for server-side processing.

The key distinction is that GET places query data in the URL, while POST typically carries submitted data in the request body. POST does not encrypt data by itself; HTTPS is required to protect credentials during transmission.

3.2 Analyze which method is more suitable for login and why.

The POST method is more suitable for login authentication for the following reasons:

1. Reduced URL exposure: Credentials are not included in the URL.
2. Better privacy: Passwords are less likely to appear in browser history or URL-based logs.
3. Appropriate purpose: POST is designed for submitting information that the server processes.
4. Improved handling: The server can validate submitted credentials and establish an authenticated session.

However, POST alone does not make authentication secure. The login request must use HTTPS and appropriate server-side security controls.

3.3 Identify security risks in using GET for authentication.

1. URL exposure: Usernames and passwords may be visible in the address bar.
2. Browser history: Credentials may remain in the browser's history.
3. Server and proxy logs: Systems that log request URLs may record sensitive information.
4. Accidental sharing: A URL containing credentials may be copied, bookmarked, or shared.
5. Referrer leakage: Sensitive URL parameters may be exposed through referrer information in certain circumstances, depending on browser behavior and policy.
6. Monitoring exposure: Browser extensions, analytics systems, and other URL-processing tools may capture sensitive parameters.

These risks can result in credential disclosure and unauthorized account access.

3.4 Suggest best practices for secure login implementation.

- Use HTTPS: Encrypt communication between the browser and server using TLS.
- Use POST: Submit login credentials in the request body rather than as URL parameters.
- Validate credentials on the server: Never rely only on client-side validation.
- Hash passwords securely: Store passwords using a password-hashing algorithm such as Argon2id or bcrypt, with appropriate parameters and salts.
- Secure session management: Use unpredictable session identifiers and cookies with Secure,

HttpOnly, and an appropriate SameSite attribute.

- Prevent brute-force attacks: Apply rate limiting and monitor repeated failed login attempts.
- Avoid logging passwords: Never store plaintext credentials in application logs.
- Implement multi-factor authentication (MFA): Add another verification step when appropriate.

Conclusion: POST is preferable to GET for login because it avoids placing credentials in the URL. Secure authentication also requires HTTPS, password hashing, server-side validation, and secure session management.

4: HTML Code Inspection

Analyze the following HTML snippet:

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph
<img src="image.jpg">
</body>
</html>
```

Questions:

1. **Identify missing closing tags.**
2. **Analyze issues with the <img>tag.**
3. **Determine how browsers will handle these errors.**
4. **Suggest a corrected version of the code.**

4.1 Identify missing closing tags.

Two elements are missing explicit closing tags:

- <h1> requires </h1>.
- requires .

The <img> element does not require a closing tag because it is a void element in HTML5.

4.2 Analyze issues with the <img> tag.

Given image tag:

<img src="image.jpg">

The main issues are:

1. Missing alt attribute: Alternative text is important for screen-reader accessibility and can be displayed when the image cannot load.
2. Image path: The file image.jpg must exist at the specified location. Otherwise, the browser cannot display it.
3. Missing dimensions: Specifying width and height can help the browser reserve space and reduce layout shifts.

Corrected image tag:

<img src="image.jpg" alt="Sample image" width="300" height="200">

The dimensions should reflect the actual image's aspect ratio and intended display size.

4.3 Determine how browsers will handle these errors.

- Browsers generally attempt to recover from missing closing tags using HTML parsing rules.
- The heading and paragraph may still render correctly because the browser can infer the end of certain elements.
- If image.jpg is missing or inaccessible, the browser generally displays a broken-image indicator; meaningful alternative text can help communicate the image's purpose.
- The page may still appear in the browser, but malformed HTML can create unexpected DOM structures and accessibility problems.

Therefore, successful rendering does not necessarily mean the HTML source is correctly structured.

4.4 Suggest a corrected version of the code.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Sample</title>
</head>
<body>
  <h1>Welcome</h1>
  <p>This is a paragraph.</p>
  <img src="image.jpg" alt="Sample image" width="300" height="200">
</body>
</html>
```

5: Browser-Server Communication Flow

The following sequence describes a web request:

User enters a URL in the browser
DNS lookup occurs
HTTP request is sent
Server processes request
Response is returned
Browser renders page

Questions:

1. **Interpret each step in the communication flow.**
2. **Identify where delays can occur.**
3. **Analyze the role of DNS in the process.**
4. **Suggest optimizations to improve performance.**

5.1 Interpret each step in the communication flow.

Step 1: User enters a URL

The user types a website address, such as www.example.com, into the browser. The browser identifies the destination and prepares to request the resource.

Step 2: DNS lookup

The browser resolves the domain name to an IP address using the Domain Name System (DNS), allowing it to locate the destination server.

Step 3: HTTP request is sent

The browser establishes the required network connection and sends an HTTP request. For HTTPS, TLS secures the communication.

Step 4: Server processes the request

The web server receives the request, executes application logic, accesses a database if needed, and prepares the response.

Step 5: Response is returned

The server sends an HTTP response containing a status code, headers, and possibly the requested resource, such as HTML.

Step 6: Browser renders the page

The browser parses HTML, applies CSS, executes JavaScript, and displays the webpage. It may send additional requests for images, stylesheets, scripts, and other resources.

5.2 Identify where delays can occur.

Stage	Possible cause of delay
DNS lookup	Slow DNS resolver or uncached domain lookup
Connection establishment	TCP and TLS handshake overhead
Network transmission	High latency, packet loss, or slow internet
Server processing	Inefficient code, heavy workload, or slow database queries
Response transfer	Large files, images, or uncompressed content
Browser rendering	Large DOM, excessive JavaScript, or complex CSS

The total loading time depends on both network-related delays and the time required for the server and browser to process the content.

5.3 Analyze the role of DNS in the process.

The Domain Name System (DNS) translates human-readable domain names into IP addresses.

For example:

www.example.com → corresponding server IP address

Its main roles are:

- Name resolution: Converts domain names into IP addresses.
- Server discovery: Helps the browser locate the appropriate destination.
- Caching: Stores DNS results temporarily to reduce repeated lookup time.
- Traffic distribution: DNS records can direct clients toward different servers, depending on the service's configuration.

Without successful name resolution, the browser generally cannot connect to the server using the domain name. However, DNS caching can sometimes eliminate the need for a new lookup.

5.4 Suggest optimizations to improve performance.

1. Optimize DNS: Use reliable DNS services and appropriate caching.
2. Reuse network connections: Enable HTTP/2 or HTTP/3 where supported to improve communication efficiency.
3. Optimize server processing: Improve application code and database queries.
4. Implement caching: Cache suitable responses and static resources to avoid repeated downloads and processing.
5. Use a Content Delivery Network (CDN): Deliver static content from servers closer to users.
6. Compress resources: Use Brotli or Gzip to reduce the transfer size of text-based resources.
7. Optimize images: Compress images and use efficient formats such as WebP or AVIF.
8. Improve frontend performance: Minimize unnecessary JavaScript and CSS, defer non-critical scripts, and reduce excessive DOM complexity.

RUBRICS FOR CALCULATION:

Criteria	Weigh tage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Mark Awarded
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data	23/25
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided	23/25
Application of Concepts	20%	Effectively applies relevant concepts and tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data	18/20
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions	19/20
Clarity	10%	Presents interpretations clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand	9/10
Total	**100%**				**Overall Mark**	**92/100**